

Laboratory Work 3

Data Manipulation Language (DML)

Task 1. Generate and insert 200 random rows into the airport database

Purpose: Insert 200 random records into the tables of the airport database using PostgreSQL functions.

SQL used:

```
INSERT INTO airport
(airport_id, airport_name, country, state, city, created_at, updated_at)
SELECT
gs,
'Airport ' || gs,
'Country' || (1 + floor(random() * 10))::int,
'State' || (1 + floor(random() * 20))::int,
'City' || (1 + floor(random() * 50))::int,
CURRENT_TIMESTAMP - (random() * interval '1000 days'),
CURRENT_TIMESTAMP
FROM generate_series(1, 200) AS gs;
```

Simple explanation: The task requires random data to be inserted into the airport database tables. `generate_series()` creates a sequence of IDs, `random()` creates different values, and `||` joins text with numbers. The data was inserted while respecting the relationships between the tables.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' shows the 'airport' table under the 'public' schema. The 'Columns' tab is selected, showing the following columns: `airport_id` (integer), `airport_name` (character varying), `country` (character varying), `state` (character varying), `city` (character varying), `created_at` (timestamp without time zone), and `updated_at` (timestamp without time zone). The main pane displays the SQL query used to insert 200 random rows. The 'Data Output' pane at the bottom shows the first 5 rows of the result set.

airport_id [PK] integer	airport_name character varying (50)	country character varying (50)	state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	Airport 1	Country1	State13	City31	2026-04-26 15:53:25.540634	2026-09-26 16:53:05.895706
2	Airport 2	Country9	State15	City6	2025-12-26 12:36:31.41838	2026-09-26 16:53:05.895706
3	Airport 3	Country4	State6	City47	2025-10-02 06:21:19.725954	2026-09-26 16:53:05.895706
4	Airport 4	Country9	State12	City38	2024-02-21 20:02:21.598685	2026-09-26 16:53:05.895706
5	Airport 5	Country1	State14	City44	2025-12-27 23:23:08.767674	2026-09-26 16:53:05.895706

Total rows: 200 Query complete 00:00:00.056

Figure 1. 200 generated airport rows in pgAdmin 4

Task 2. Add KazAir

Purpose: Insert a new airline into the airline table.

SQL used:

```
INSERT INTO airline
(airline_id, airline_code, airline_name, airline_country, created_at, updated_at)
VALUES
(1, 'KAZ', 'KazAir', 'Kazakhstan', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Simple explanation: INSERT adds a new record. We provide values for all required columns. The airline has code KAZ, name KazAir, and country Kazakhstan.

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database structure, including the 'airline' table with its columns: airline_id (integer), airline_code (character varying), airline_name (character varying), airline_country (character varying), created_at (timestamp without time zone), and updated_at (timestamp without time zone). The 'airline' table is selected, and its columns are listed. The main query editor displays the following SQL script:

```
-- 1
INSERT INTO airport
(airport_id, airport_name, country, state, city, created_at, updated_at)
SELECT
gs,
'Airport' || gs,
'Country' || (1+floor(random() * 10))::int,
'State' || (1+floor(random() * 20))::int,
'City' || (1+floor(random() * 50))::int,
CURRENT_TIMESTAMP - (random() * interval '1000 days'),
CURRENT_TIMESTAMP
FROM generate_series(1, 200) AS gs;

-- 2
SELECT * FROM airport;

-- 3
INSERT INTO airline
(airline_id, airline_code, airline_name, airline_country, created_at, updated_at)
VALUES
(1, 'KAZ', 'KazAir', 'Kazakhstan', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

The 'Data Output' pane at the bottom shows the results of the queries. The first query (SELECT * FROM airport) returned 200 rows. The second query (SELECT * FROM airline) returned 1 row, which is the newly inserted record:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
1	KAZ	KazAir	Kazakhstan	2026-09-26 17:24:57.057266	2026-09-26 17:24:57.057266

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.054'.

Figure 2. Airline table after the initial insert

Task 3. Update KazAir country

Purpose: Change the country of KazAir from Kazakhstan to Turkey.

SQL used:

```
UPDATE airline
SET airline_country = 'Turkey'
WHERE airline_name = 'KazAir';
```

Simple explanation: UPDATE changes existing data. SET gives the new value, while WHERE selects only the KazAir row. The result shows Turkey for KazAir.

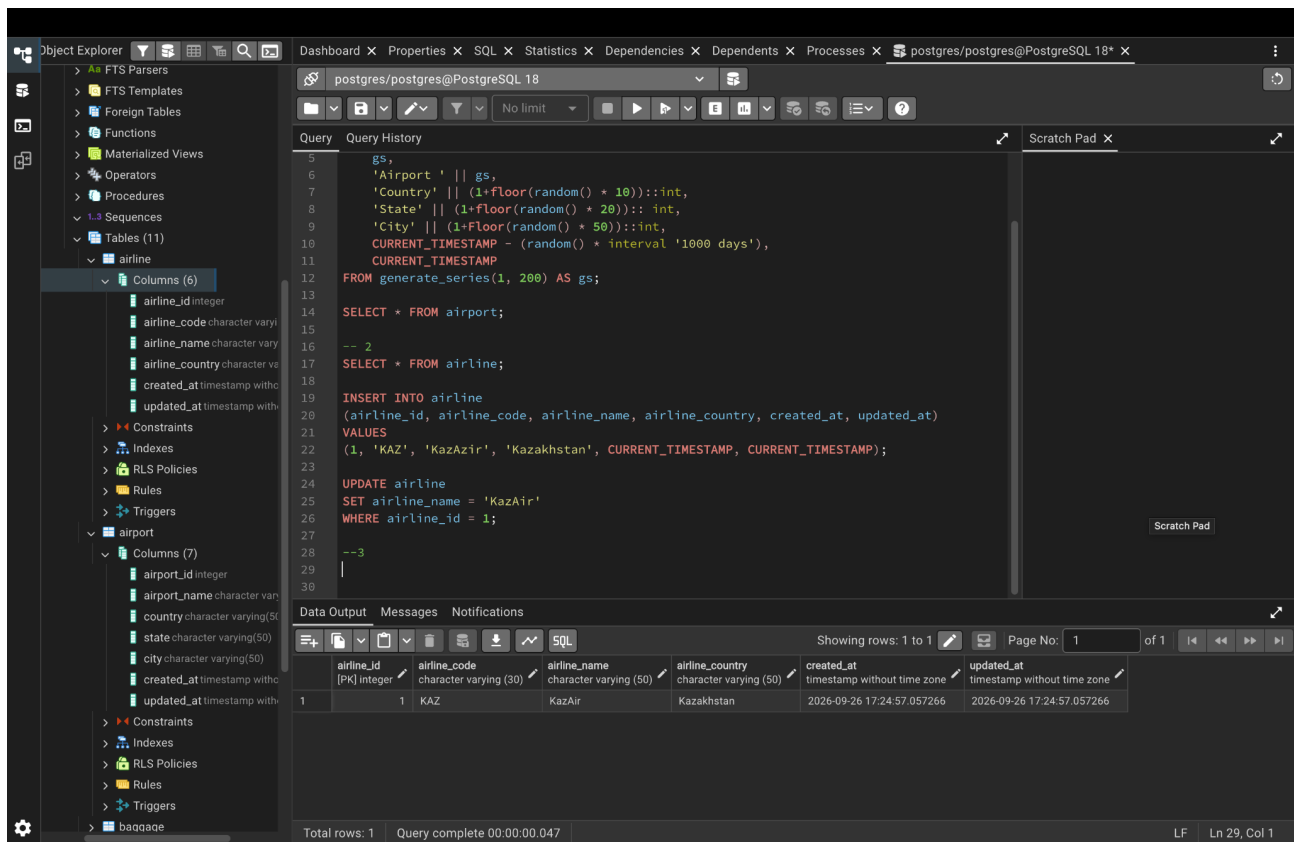


Figure 3. KazAir after the country update

Task 4. Add three airlines

Purpose: Insert three airline records in one INSERT statement.

SQL used:

```
INSERT INTO airline
(airline_id, airline_code, airline_name, airline_country, created_at, updated_at)
VALUES
(2, 'AEE', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
(3, 'FLH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
(4, 'FLF', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Simple explanation: One INSERT statement can add several rows. Here, three airlines are added at the same time. The result shows four airlines in total, including KazAir.

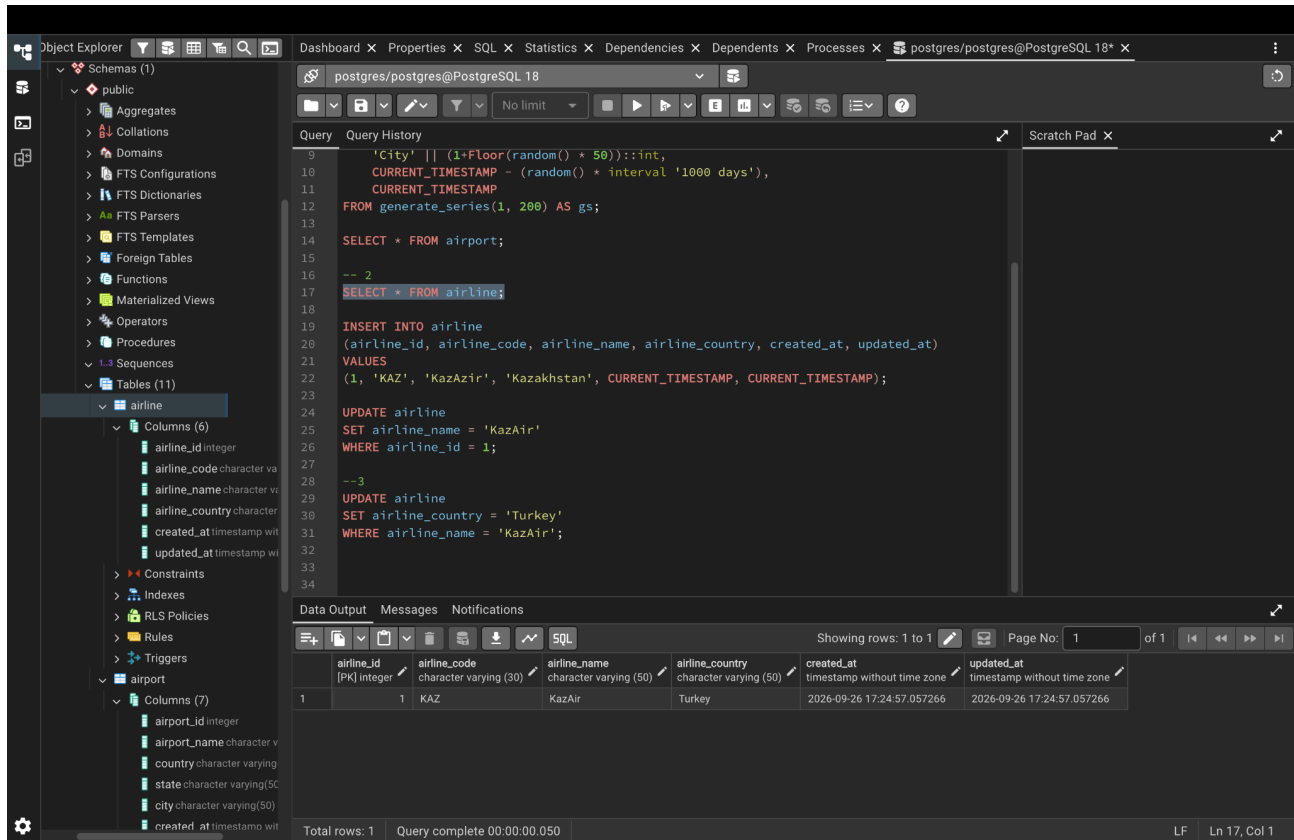


Figure 4. Four airlines in the table

Task 5. Delete flights arriving in 2024

Purpose: Remove flights whose scheduled arrival time is in the year 2024.

SQL used:

```

DELETE FROM flights
WHERE sch_arrival_time >= '2024-01-01'
AND sch_arrival_time < '2025-01-01';

```

Simple explanation: DELETE removes rows. The two conditions create a date range from January 1, 2024 up to, but not including, January 1, 2025. At the time of execution there were no matching flights, so PostgreSQL returned DELETE 0.

The screenshot displays the PostgreSQL Enterprise Developer interface. The left sidebar shows the 'airline' table selected under the 'public' schema. The main pane shows a series of SQL queries executed in the 'airline' table. The 'Data Output' pane at the bottom shows the results of the queries, which are 4 rows of data inserted into the 'airline' table.

```

17 SELECT * FROM airline;
18
19 INSERT INTO airline
20 (airline_id, airline_code, airline_name, airline_country, created_at, updated_at)
21 VALUES
22 (1, 'KAZ', 'KazAir', 'Kazakhstan', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
23
24 UPDATE airline
25 SET airline_name = 'KazAir'
26 WHERE airline_id = 1;
27
28 --3
29 UPDATE airline
30 SET airline_country = 'Turkey'
31 WHERE airline_name = 'KazAir';
32
33 --4
34 INSERT INTO airline
35 (airline_id, airline_code, airline_name, airline_country, created_at, updated_at)
36 VALUES
37 (2, 'AEE', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
38 (3, 'FLH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
39 (4, 'FLF', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
40
41 SELECT * FROM airline;
42

```

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
1	KAZ	KazAir	Turkey	2026-09-26 17:24:57.057266	2026-09-26 17:24:57.057266
2	AEE	AirEasy	France	2026-09-26 17:57:01.267085	2026-09-26 17:57:01.267085
3	FLH	FlyHigh	Brazil	2026-09-26 17:57:01.267085	2026-09-26 17:57:01.267085
4	FLF	FlyFly	Poland	2026-09-26 17:57:01.267085	2026-09-26 17:57:01.267085

Total rows: 4 Query complete 00:00:00.090

Figure 5. Task 5 execution result: DELETE 0

Task 6. Increase ticket prices by 15%

Purpose: Increase every ticket price in the booking table by 15 percent.

SQL used:

```

UPDATE booking
SET ticket_price = ticket_price * 1.15;

```

Simple explanation: The expression `ticket_price * 1.15` keeps the old price and adds 15 percent. At the time of execution there were no booking rows to update, so the result was UPDATE 0.

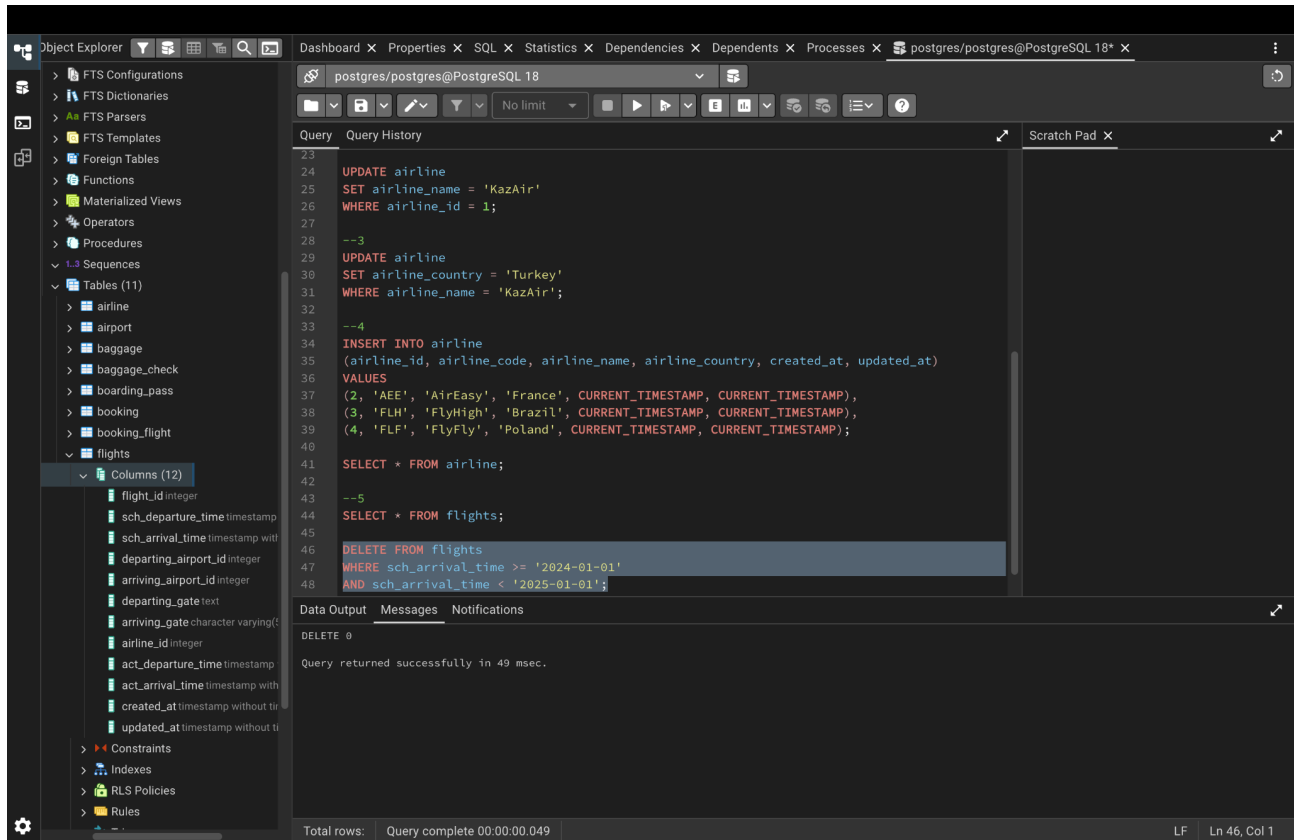


Figure 6. Task 6 execution result: UPDATE 0

Task 7. Delete tickets below 10,000

Purpose: Delete bookings where the ticket price is less than 10,000.

SQL used:

```
DELETE FROM booking
WHERE ticket_price < 10000;
```

Simple explanation: The WHERE condition selects only tickets cheaper than 10,000. There were no matching booking rows at the time, so PostgreSQL returned DELETE 0.

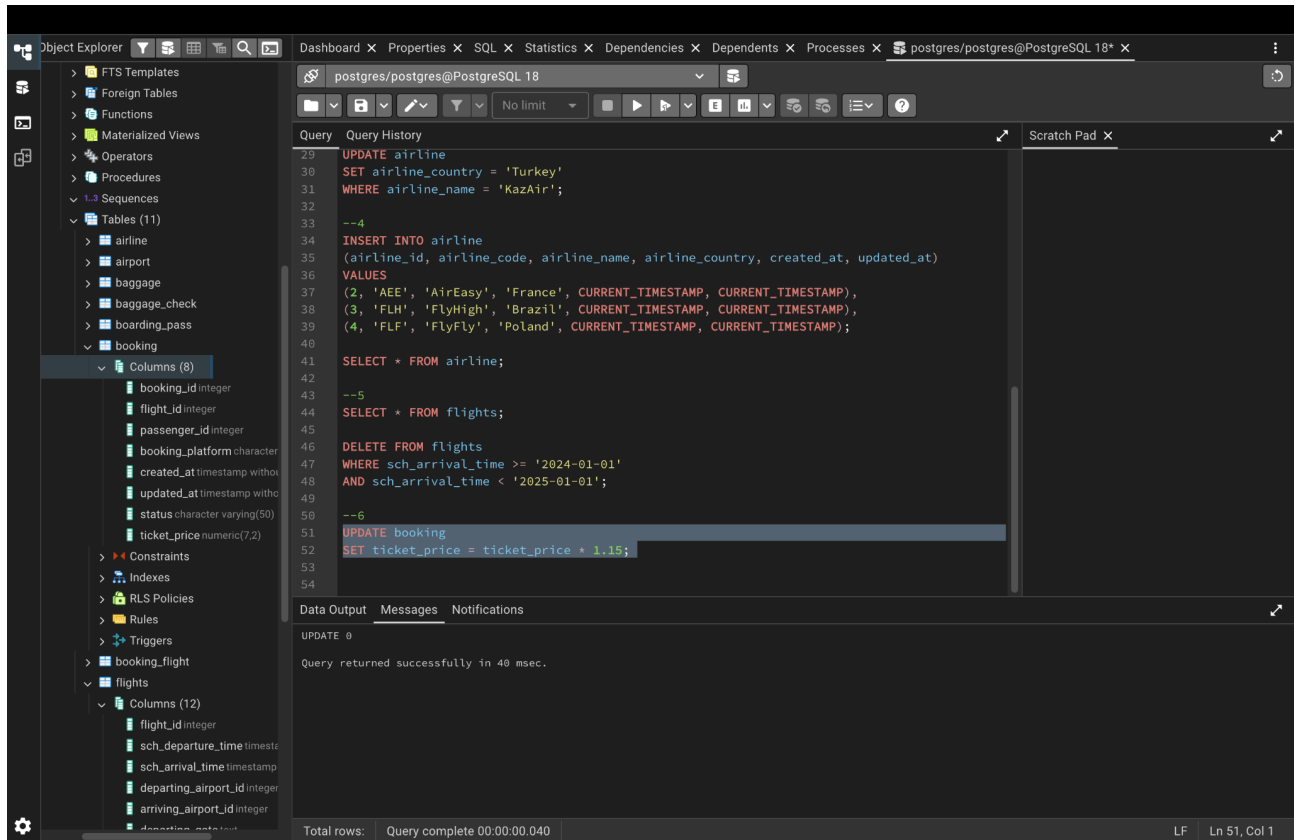


Figure 7. Task 7 execution result: DELETE 0

Task 8. Set NULL airline codes to UNK

Purpose: Replace missing airline codes with the default text UNK.

SQL used:

```
UPDATE airline
SET airline_code = 'UNK'
WHERE airline_code IS NULL;
```

Simple explanation: IS NULL checks for missing values. If an airline code were NULL, it would be changed to UNK. In the current database the airline_code column is NOT NULL, so no rows matched and the result was UPDATE 0.

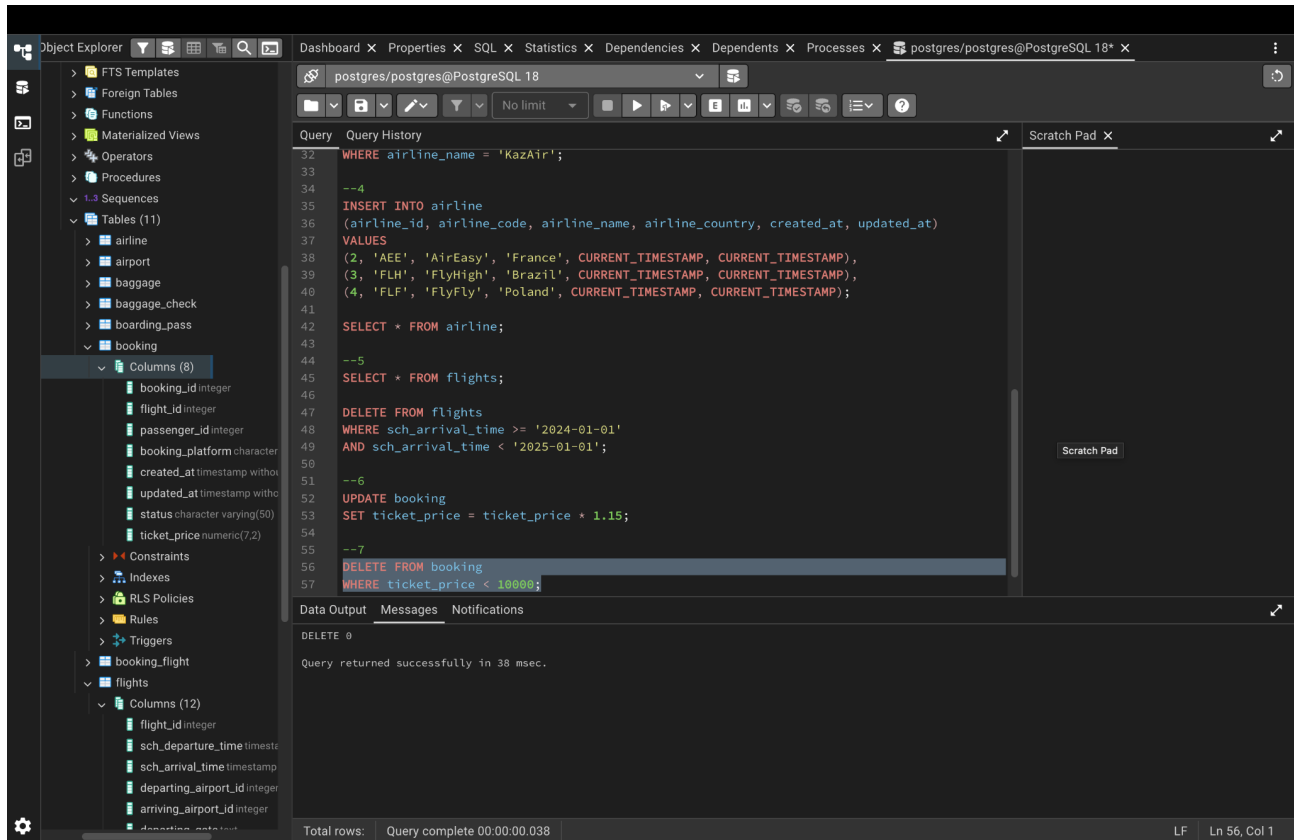


Figure 8. Task 8 execution result: UPDATE 0

Task 9. Delete old unchecked baggage checks

Purpose: Delete baggage checks created before June 1, 2023 when the result is Not checked.

SQL used:

```
DELETE FROM baggage_check
WHERE created_at < '2023-06-01'
AND check_result = 'Not checked';
```

Simple explanation: The query uses two conditions: an old creation date and the result Not checked. No row matched both conditions during the execution, so the result was DELETE 0.

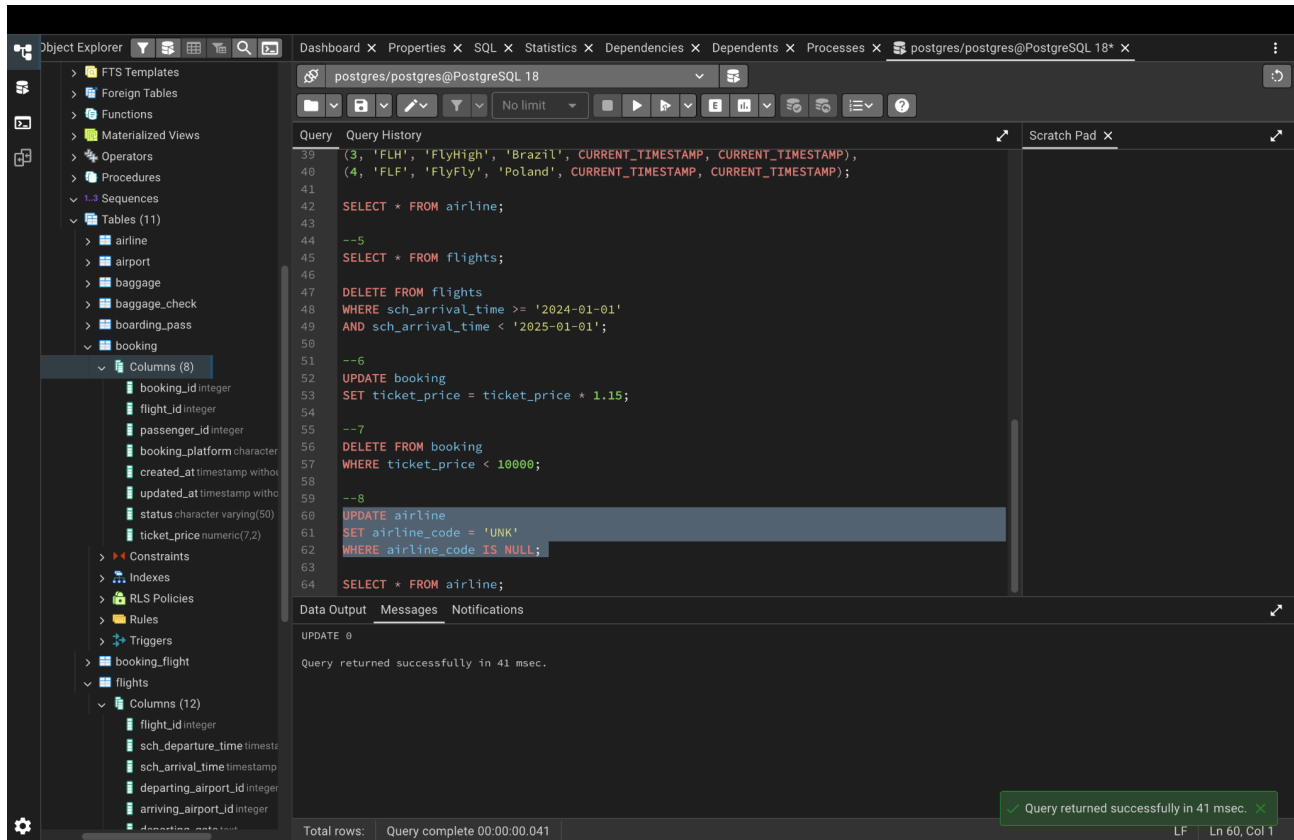


Figure 9. Task 9 execution result: DELETE 0

Task 10. Delete airports with NULL state and selected cities

Purpose: Delete airports where state is NULL and the city is Mlawe or Kepuh.

SQL used:

```
DELETE FROM airport
WHERE state IS NULL
AND city IN ('Mlawe', 'Kepuh');
```

Simple explanation: IS NULL checks for a missing state, and IN checks several possible city values. In the current database the state column is defined as NOT NULL, so rows with state = NULL cannot exist. Therefore the query correctly returned DELETE 0.

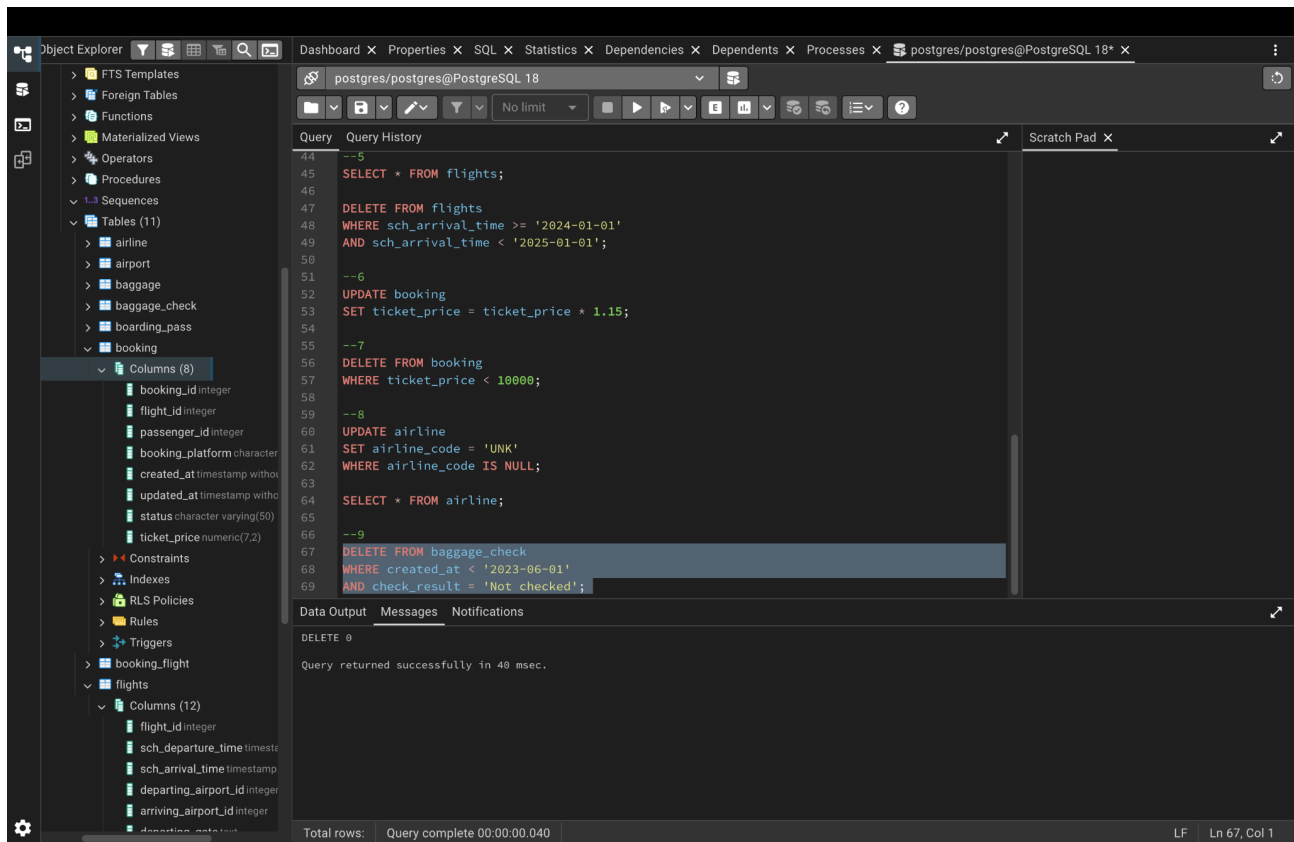


Figure 10. Task 10 execution result: DELETE 0

Task 11. Insert a baggage check and return its ID

Purpose: Insert a new unchecked baggage inspection and immediately return its ID and creation time.

SQL used:

```

INSERT INTO baggage_check
(baggage_check_id, check_result, created_at, updated_at, booking_id, passenger_id)
VALUES
(1, 'Not checked', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 1, 1)
RETURNING baggage_check_id, created_at;

```

Simple explanation: INSERT adds the baggage check. RETURNING sends selected values back immediately after the insert. The result returned baggage_check_id = 1 and the current created_at timestamp.

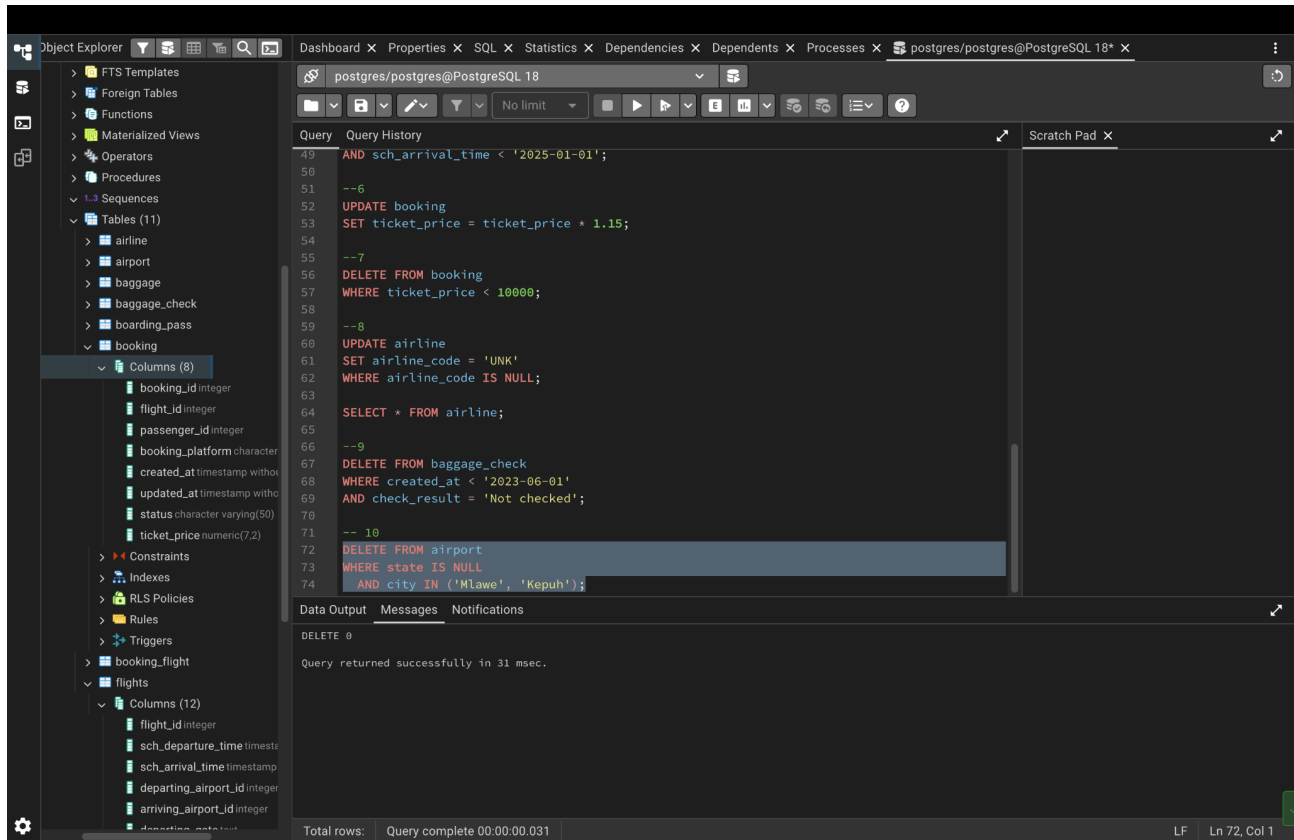


Figure 11. Supporting data for the Task 11 record

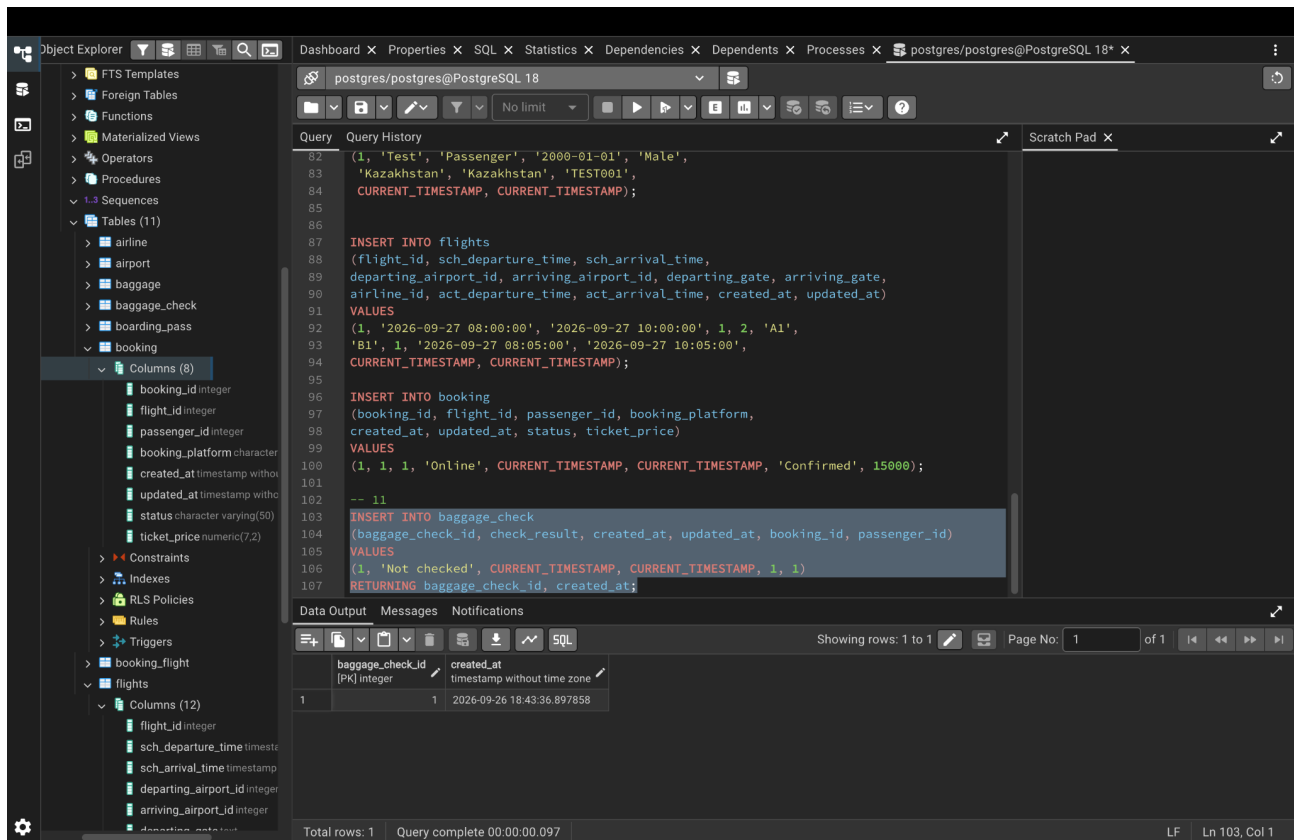


Figure 12. Task 11 result returned by RETURNING

Task 12. Uppercase all airline countries

Purpose: Convert all country names in the airline table to uppercase.

SQL used:

```
UPDATE airline
SET airline_country = UPPER(airline_country);
```

Simple explanation: UPPER converts text to capital letters. The UPDATE applies the function to every airline country. The result showed UPDATE 4 for the four airlines existing at that stage.

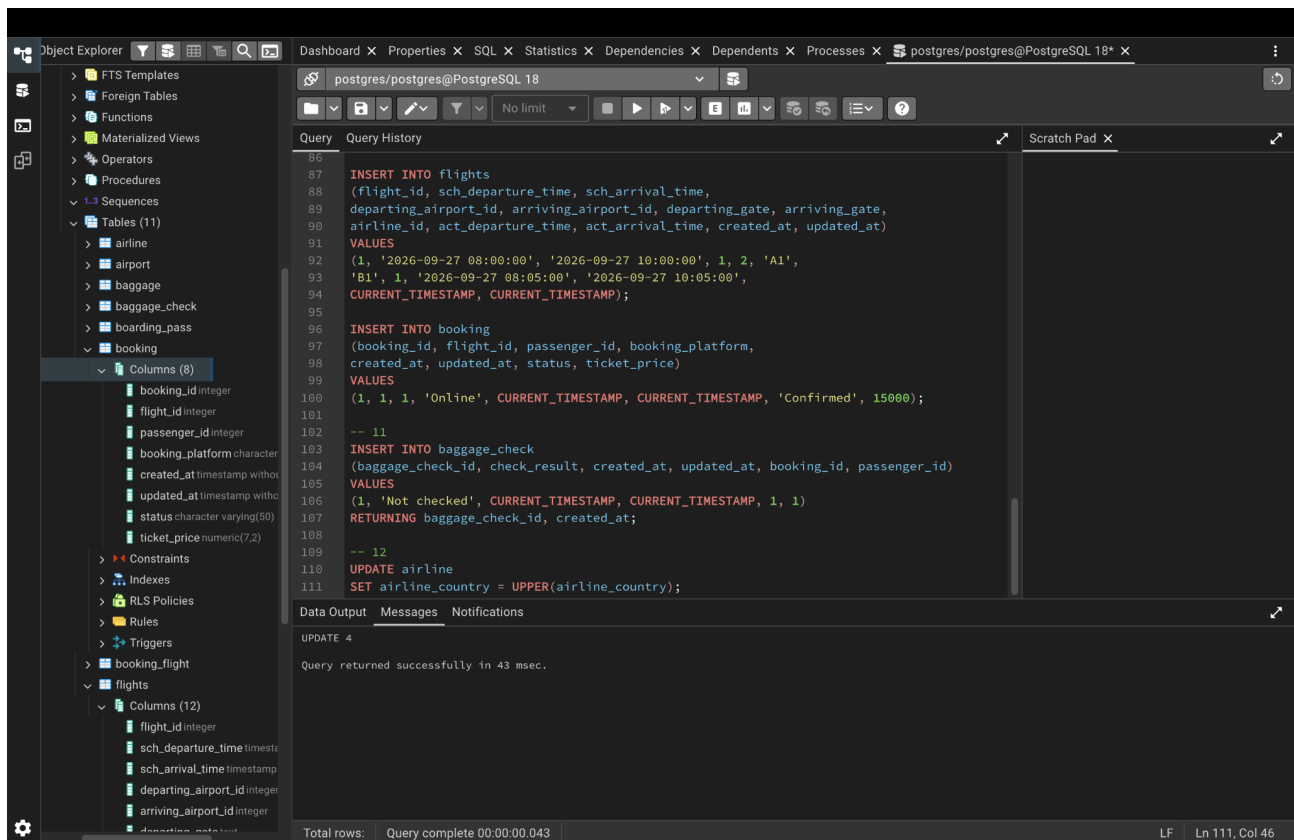


Figure 13. Task 12 execution result: UPDATE 4

Task 13. Update airline ID 5

Purpose: Set airline ID 5 to Global Airways, United Kingdom, and update its date.

SQL used:

```
UPDATE airline
SET airline_name = 'Global Airways',
    airline_country = 'United Kingdom',
    updated_at = CURRENT_DATE
WHERE airline_id = 5;
```

Simple explanation: The WHERE condition selects airline ID 5. SET changes its name, country, and update date. A record with ID 5 was added as supporting data before this update, so the task could be demonstrated successfully.

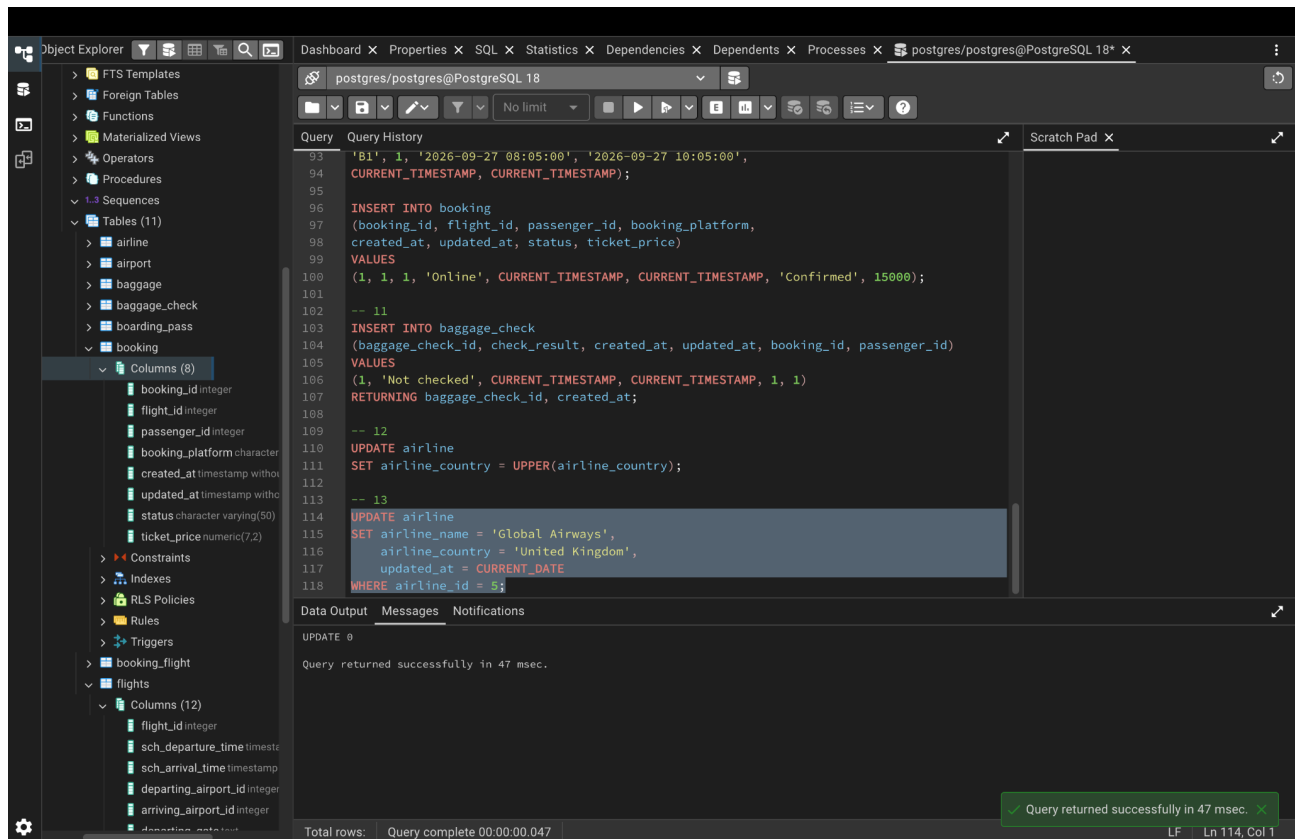


Figure 14. Supporting data and Task 13 execution

Task 14. Set airport state to Capital District

Purpose: Set the state of airports in Astana, London, and Tokyo to Capital District.

SQL used:

```
UPDATE airport
SET state = 'Capital District'
WHERE city IN ('Astana', 'London', 'Tokyo');
```

Simple explanation: IN allows several city values in one condition. The three supporting airport rows were added before the update. The result was UPDATE 3, so all three matching airports were changed.

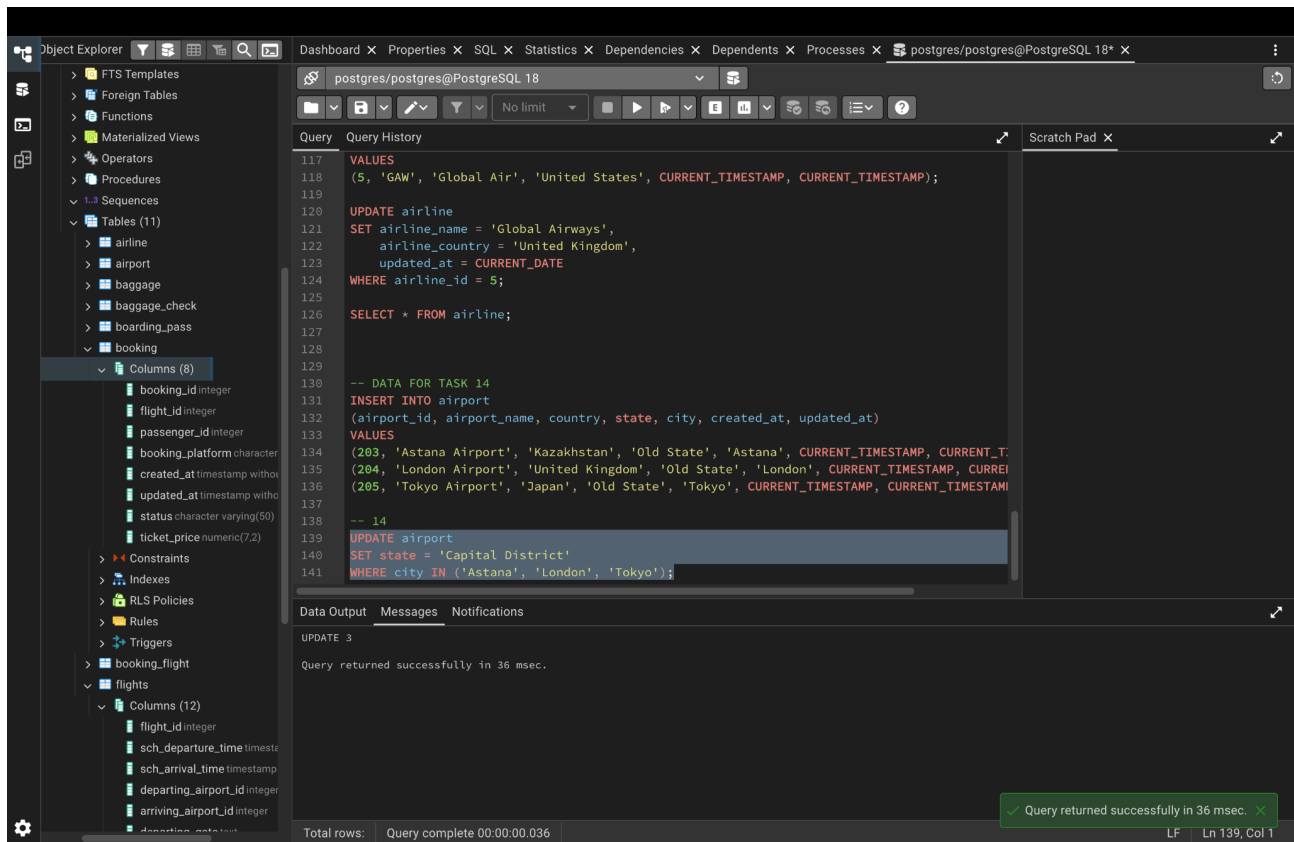


Figure 15. Task 14 result: UPDATE 3

Task 15. Mark March 2024 baggage checks as Checked

Purpose: Change baggage checks from Not checked to Checked when they were created in March 2024.

SQL used:

```
UPDATE baggage_check
SET check_result = 'Checked'
WHERE created_at >= '2024-03-01'
AND created_at < '2024-04-01'
AND check_result = 'Not checked';
```

Simple explanation: The date conditions select only March 2024. The last condition selects only unchecked records. The supporting record was created on March 15, 2024, so the query returned UPDATE 1.

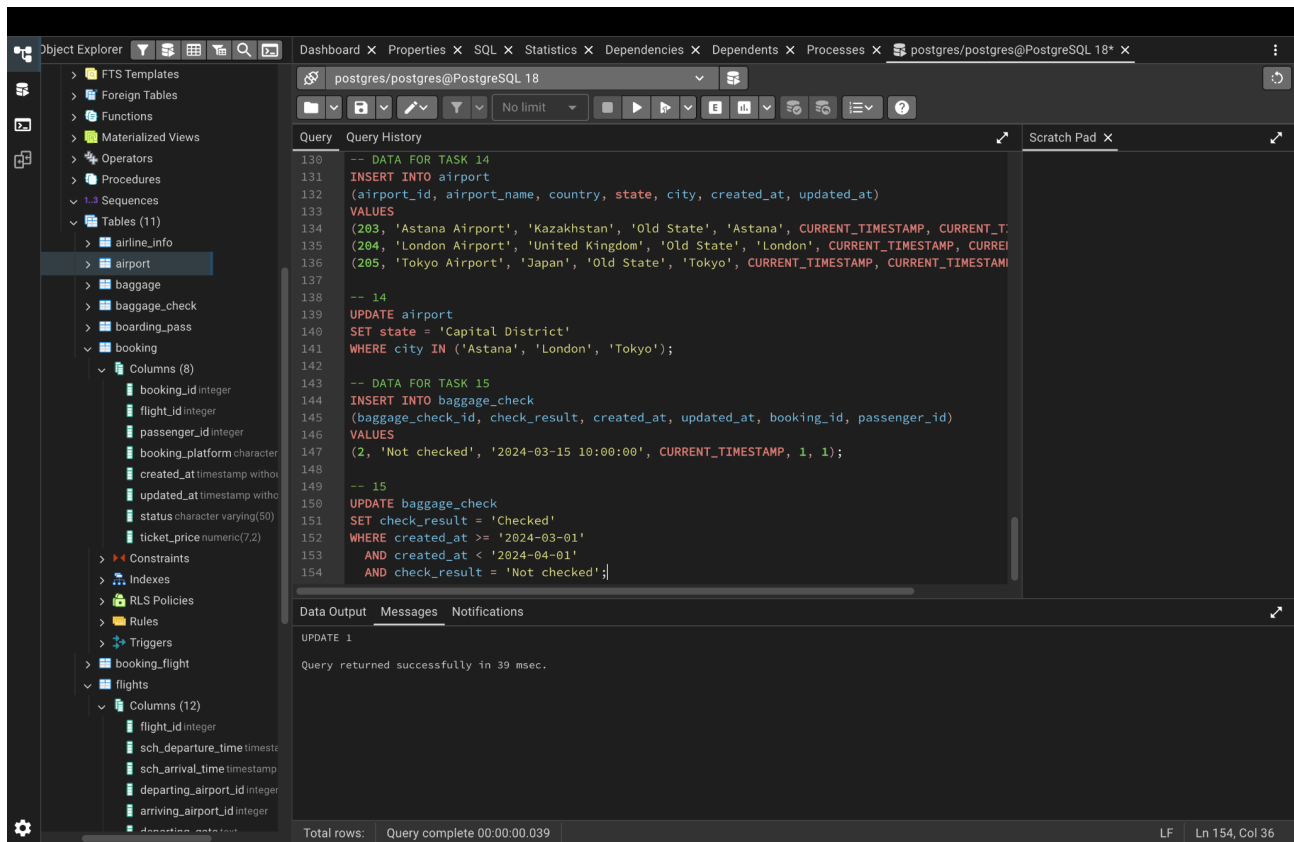


Figure 16. Supporting data and Task 15 execution

DML Explanation

Note: UPDATE 0 or DELETE 0 does not mean that the SQL query is incorrect. It means that no current rows matched the condition. The result depends on the current dataset and table constraints.

DML (Data Manipulation Language) is a part of SQL used to work with the data stored inside database tables. It allows us to add, change, and remove records.

Main DML commands

Command	Purpose	Example
INSERT	Adds new rows to a table.	INSERT INTO airline ...
UPDATE	Changes existing rows.	UPDATE airline SET ...
DELETE	Removes rows from a table.	DELETE FROM flights WHERE ...

Important SQL ideas used in this lab

- **VALUES:** Provides the values that will be inserted into a table.
- **SET:** Specifies the new values in an UPDATE statement.
- **WHERE:** Chooses which rows should be changed or deleted.
- **IS NULL:** Checks whether a value is missing.
- **IN:** Checks whether a value is one of several listed values.
- **UPPER():** Converts text to uppercase letters.
- **RETURNING:** Returns selected values from rows affected by an INSERT, UPDATE, or DELETE.
- **CURRENT_TIMESTAMP:** Gets the current date and time.
- **CURRENT_DATE:** Gets the current date.

INSERT, UPDATE and DELETE in simple words

- INSERT means: put new data into the table.
- UPDATE means: change data that is already in the table.
- DELETE means: remove data from the table.
- A WHERE condition is very important for UPDATE and DELETE because it limits which rows are affected.
- Without WHERE, an UPDATE or DELETE can affect all rows in the table.

Conclusion

In this laboratory work, I practiced DML operations in PostgreSQL using pgAdmin 4. I learned how to insert new records, update existing data, delete records, use conditions, work with dates, use functions such as UPPER(), and return values after an INSERT operation. The laboratory also showed that the result of a query depends on the current data and table constraints. For example, some queries returned 0 rows because there were no matching records or because a column had a NOT NULL constraint. Overall, DML is important because it allows a database user to manage and modify the actual data stored in tables.